Sparse classification problem

Simulated example

Sparse Classification

Install HDRM package (source of data)

# Sparse Classification

**Jyotishka Data**

**2023-04-03**

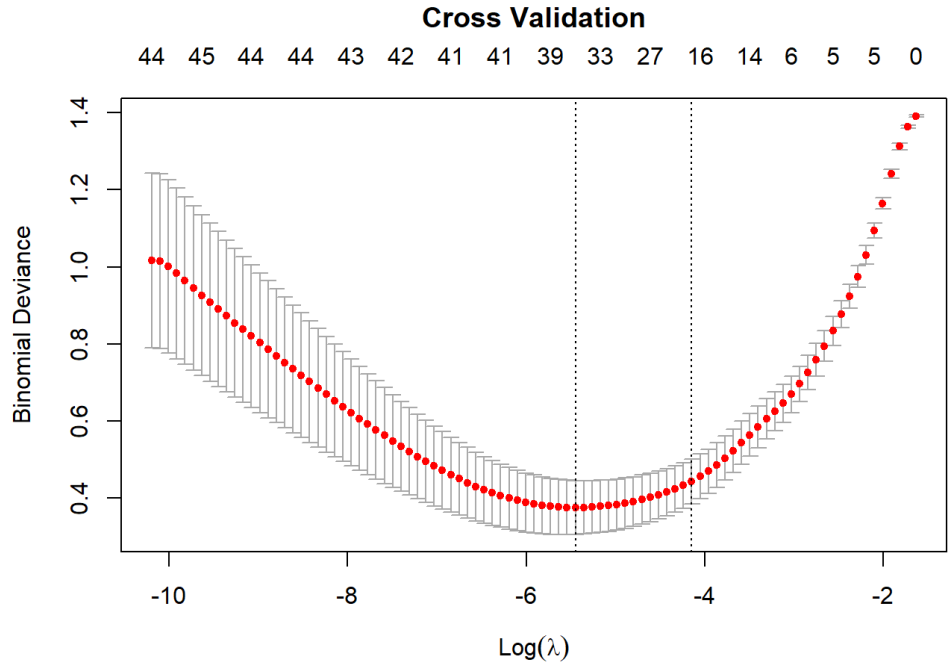
# Sparse classification problem

We will show that the same LASSO penalty can work for sparse classification problems as well.

# Simulated example

We will start with a simulated example with $\beta$ having 5 non-zeros and rest zeros, with $n = 200, p = 50$ in a logistic regression set up, and evaluate the classification performance.

```
library(Matrix)
set.seed(12345)

logit.inv <- function(x){
  1/(1+exp(-x))
}

## Generate data
n=200;p=50
p1 = 5
beta <- c(rep(3,p1),rep(0,p-p1)) ## First p1 beta non-zer
o, remaining p-p1 zeros.

x=scale(matrix(rnorm(n*p),n,p))
eps=rnorm(n,mean=0,sd=0.1)

fx = x %*% beta

y=rbinom(n, size = 1, prob= logit.inv(fx+eps))

train=sample(200,100) ## for supervised learning

library(glmnet)

## Cross validation to learn tuning parameter
cv.out <- cv.glmnet(x = x, y = y, alpha = 1, subset = tra
in, family = "binomial") # binomial for binary response
plot(cv.out)
title("Cross Validation",line=2.5)
```




```
(bestlam =cv.out$lambda.min)
```

```
## [1] 0.004299185
```

```
y.pred=predict(cv.out,type ="class",s=bestlam, newx = x[-
train,])

y.obs <- y[-train]
xtabs(~ y.obs +y.pred) ## Misclassification table.
```

```
##       y.pred
## y.obs  0  1
##     0 47  1
##     1  0 52
```

```
library(caret)
confusionMatrix(data = as.factor(y.obs), reference = as.f
actor(y.pred))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 47  1
##          1  0 52
##
##                Accuracy : 0.99
##                  95% CI : (0.9455, 0.9997)
##     No Information Rate : 0.53
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.98
##
##  Mcnemar's Test P-Value : 1
##
##             Sensitivity : 1.0000
##             Specificity : 0.9811
##          Pos Pred Value : 0.9792
##          Neg Pred Value : 1.0000
##              Prevalence : 0.4700
##          Detection Rate : 0.4700
##    Detection Prevalence : 0.4800
##       Balanced Accuracy : 0.9906
##
##        'Positive' Class : 0
##
```

# Sparse Classification

## Golub 1999 Data

This data set consists of 47 patients with acute lymphoblastic leukemia (ALL) and 25 patients with acute myeloid leukemia (AML). Each of the 72 patients had bone marrow samples obtained at the time of diagnosis. Gene expression measurements for the samples were taken using Affymetrix Hgu6800 chips, resulting in 7129 measurements per patient.

Of the two diseases, AML has a considerably worse prognosis: only 26% survive at least 5 years following diagnosis, compared to 68% for ALL.

# Install HDRM package (source of data)

**Warning: the installation of packages and libraries will take some time when you run this for the first time ever. Then you can safely change the flag so that it does not install the package every time you run the R codes.**

```
installed_flag = F

if(installed_flag == F){
  if(!require(remotes)){
  install.packages("remotes", dependencies = TRUE, repos
= 'http://cran.rstudio.com')
 }
remotes::install_github("pbreheny/hdrm")
}
```

```
library(hdrm)
downloadData(Golub1999)
attachData(Golub1999)

n = nrow(X)
# Split data into train and test sets
train_rows <- sample(1:n, n/2)
X.train <- X[train_rows, ]
X.test <- X[-train_rows, ]

y.train <- y[train_rows]
y.test <- y[-train_rows]

dim(X.train)
```

```
## [1]   36 7129
```

```
dim(X.test)
```

```
## [1]   36 7129
```

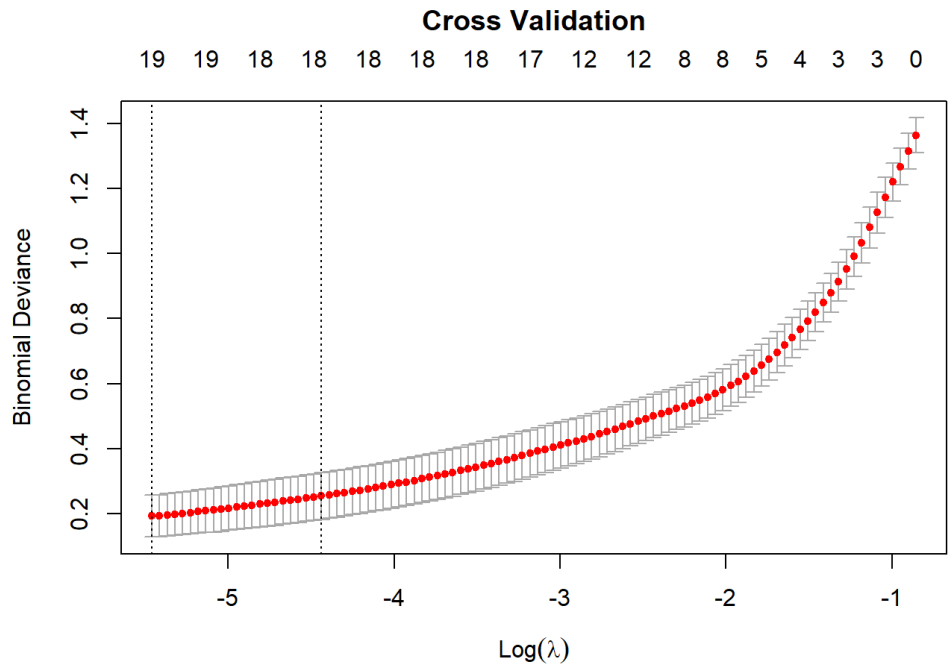
```
length(y.train)
```

```
## [1] 36
```

```
length(y.test)
```

```
## [1] 36
```

```
library(glmnet)

## Cross validation to learn tuning parameter
cv.out <- cv.glmnet(x = X.train, y = y.train, alpha = 1,
family = "binomial") # binomial for binary response
plot(cv.out)
title("Cross Validation",line=2.5)
```




```
(bestlam =cv.out$lambda.1se)
```

```
## [1] 0.01183503
```

```
y.pred=predict(cv.out,type ="class",s=bestlam, newx = X.t
est)

xtabs(~ y.test +y.pred) ## Misclassification table.
```

```
##       y.pred
## y.test ALL AML
##    ALL  25   0
##    AML   1  10
```

```
library(caret)
confusionMatrix(data = y.test, reference = as.factor(y.pr
ed))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction ALL AML
##        ALL  25   0
##        AML   1  10
##
##                Accuracy : 0.9722
##                  95% CI : (0.8547, 0.9993)
##     No Information Rate : 0.7222
##     P-Value [Acc > NIR] : 0.0001213
##
##                   Kappa : 0.9328
##
##  Mcnemar's Test P-Value : 1.0000000
##
##             Sensitivity : 0.9615
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 0.9091
##              Prevalence : 0.7222
##          Detection Rate : 0.6944
##    Detection Prevalence : 0.6944
##       Balanced Accuracy : 0.9808
##
##        'Positive' Class : ALL
##
```